Topic 4: Working with JSON Objects, Arrays & Expressions

> **Institution:** Tayana Academy | LMS Assessment
> **Topic:** N8N Hands-On Beginner | Topic 4: Working with JSON Objects, Arrays & Expressions
> **Workflow Name:** `Topic4_JSON_Data_Assessment`
> **Target Audience:** Trainee / Employee
> **Difficulty Level:** Beginner
> **Platform Version:** n8n version 1.0.0 or higher
> **Student:** Abhishek Nikam (`abhishek.n@tayanasolutions.com`)
> **Repository:** n8n-assessments

---

1. Assessment Overview & Objectives

This practical assessment evaluates the learner's capability to generate, inspect, navigate, and transform complex hierarchical JSON structures in n8n. The assessment focuses on distinguishing JSON objects from arrays, using dot notation and zero-based bracket indexing in n8n expressions, preserving native data types across transformations, and delivering a clean, simplified output schema.

Key Deliverables in this Package:

1. `README.md` : Comprehensive assessment documentation covering architecture, JavaScript mock generation, expression mechanics, multi-view inspection, and LMS submission notes.
2. `Topic4_JSON_Data_Assessment.json` : Official exportable n8n workflow file ready to import into any n8n v1+ instance.
3. `README.pdf` : Print-ready, professionally styled PDF document generated directly from this documentation.

---

2. Workflow Architecture & Data Pipeline

The workflow executes in a linear 3-node sequence from left to right:

```
flowchart LR
    A["🟢 Manual Trigger<br/>('When clicking Test workflow')"] --> B["💻 Code Node<br/>('Mock Neste
    B --> C["✂️ Edit Fields (Set)<br/>('Extract Values')"]

    subgraph Step2["Complex Source JSON (Code Node)"]
        B1["customer: { name, address: { city } }<br/>orders: [ { total }, ... ]<br/>membership: 'Go
    end

    subgraph Step3["Flattened Output (Edit Fields)"]
        C1["customerName: 'Jane Doe'<br/>cityOfCustomer: 'San Francisco'<br/>firstOrderTotal: 149.99
    end

    B -.-> Step2
    C -.-> Step3

    classDef trigger fill:#10b981,stroke:#047857,stroke-width:2px,color:#fff;
    classDef code fill:#f59e0b,stroke:#d97706,stroke-width:2px,color:#fff;
    classDef set fill:#6366f1,stroke:#4338ca,stroke-width:2px,color:#fff;
    class A trigger;
    class B code;
    class C set;
```

---

3. Node Specifications & Detailed Configuration

Node 1: Manual Trigger

- **Node Label:** `When clicking 'Test workflow'`
- **Node Type:** `n8n-nodes-base.manualTrigger` (typeVersion: `1`)
- **Purpose:** Entry trigger for controlled execution and testing on the editor canvas.

Node 2: Code Node (`Mock Nested JSON Data`)

- **Node Label:** `Mock Nested JSON Data`
- **Node Type:** `n8n-nodes-base.code` (typeVersion: `2`)
- **Language:** JavaScript
- **Code Snippet:**

```
return [
  {
    json: {
      customer: {
        id: "CUST-8492",
        name: "Jane Doe",
        email: "jane.doe@example.com",
        address: {
          street: "123 Market Street",
          city: "San Francisco",
          state: "CA",
          postalCode: "94105",
          country: "USA"
        }
      },
      orders: [
        {
          orderId: "ORD-1001",
          item: "Ergonomic Keyboard",
          quantity: 1,
          total: 149.99,
          status: "completed"
        },
        {
          orderId: "ORD-1002",
          item: "Wireless Mouse",
          quantity: 2,
          total: 50.00,
          status: "shipped"
        }
      ],
      membership: "Gold",
      status: "active"
    }
  }
];
```

Node 3: Edit Fields (Set) Node (`Extract Values`)

- **Node Label:** `Extract Values`
- **Node Type:** `n8n-nodes-base.set` (typeVersion: `3.4`)
- **Include Other Input Fields:** `false` (produces a clean, simplified output containing only the extracted properties)
- **Extracted Fields & Expressions:**

Output Field Name	Data Type	Expression Syntax	Value Source / Path	Result Value
`customerName`	`String`	`={{ $json.customer.name }}`	Dot notation into object `customer`	`"Jane Doe"`
`cityOfCustomer`	`String`	`={{ $json.customer.address.city }}`	Nested dot notation into `customer.address`	`"San Francisco"`
`firstOrderTotal`	`Number`	`={{ $json.orders[0].total }}`	Array index `[0]` into `orders` array	`149.99`

---

4. Assessment Tasks & Step-by-Step Walkthrough

Exercise 1 – Create and Name the Assessment Workflow

- Created a fresh blank workflow canvas.
- Renamed the workflow exactly to:

```
Topic4_JSON_Data_Assessment
```

- Added the `Manual Trigger` node and saved the workflow before proceeding.

Exercise 2 – Generate Nested Customer JSON Data

- Placed a `Code` node immediately following the trigger and renamed it to **`Mock Nested JSON Data`**.
- Implemented JavaScript code returning:
  1. A nested `customer` object containing customer attributes and a nested `address` sub-object.
  2. An `orders` array containing two order objects with `orderId`, `item`, `quantity`, `total`, and `status`.
  3. Single-value fields: `membership` (`"Gold"`) and `status` (`"active"`).
- Executed node and validated that n8n parses the structure as valid JSON.

Exercise 3 – Inspect Objects, Arrays, and Data Types

- Opened output panel of `Mock Nested JSON Data` across all three view modes:
  - **JSON View:** Inspected curly braces `{}` representing key-value objects and square brackets `[]` representing indexed arrays.
  - **Table View:** Observed nested structures rendered as collapsed object pills (`[Object]`, `[Array]`).
  - **Schema View:** Verified inferred types: `customer` (Object), `customer.name` (String), `orders` (Array of Objects), `orders[].total` (Number).
- Confirmed zero-based array indexing: the first order (`ORD-1001`) resides at index `0`.

Exercise 4 – Extract Nested Values Using Expressions

- Added an `Edit Fields (Set)` node and renamed it to **`Extract Values`**.
- Configured three targeted attributes using n8n expression mode:
  1. `customerName`: `={{ $json.customer.name }}` (type: String)
  2. `cityOfCustomer`: `={{ $json.customer.address.city }}` (type: String)
  3. `firstOrderTotal`: `={{ $json.orders[0].total }}` (type: Number)
- Verified that all fields utilize dynamic expression binding rather than hardcoded string literals.

Exercise 5 – Execute, Validate, Document, and Submit

- Executed the complete workflow end-to-end via **Test workflow**.
- Verified all three nodes completed successfully with green checkmarks.
- Confirmed the final output schema contains exactly the three extracted fields with correct native types.
- Exported workflow JSON file: `Topic4_JSON_Data_Assessment.json`.
- Generated publication PDF: `README.pdf`.
- Updated repository portfolio at `3OOM/n8n-assessments`.

---

5. Final Output Payload & Multi-View Verification

Final Transformed Output JSON

```
[
  {
    "customerName": "Jane Doe",
    "cityOfCustomer": "San Francisco",
    "firstOrderTotal": 149.99
  }
]
```

Multi-View Inspection Matrix

View Mode	What Evaluator Observes	Verification Criteria
Table View	3 columns: `customerName`, `cityOfCustomer`, `firstOrderTotal`. Single row displaying `Jane Doe | San Francisco | 149.99`.	Clean flattened representation without nested object clutter.
JSON View	Clean key-value object containing exactly the 3 extracted properties matching the required keys.	Exact match: values dynamically derived from source data.
Schema View	• `customerName` : `string` • `cityOfCustomer` : `string` • `firstOrderTotal` : `number`	Confirms strict preservation of String and Number primitive types.

---

6. Technical Concepts: Navigating Hierarchical JSON

1. Objects (`{}`) vs. Arrays (`[]`)

- **JSON Objects (`{}`):** Unordered collections of key-value pairs accessed via keys using dot notation (e.g. `$json.customer.name`).
- **JSON Arrays (`[]`):** Ordered collections of values accessed via numerical position using bracket indexing (e.g. `$json.orders[0]`).

2. Dot Notation for Deeply Nested Objects

- Dot notation traverses successive hierarchy layers:
  `$json` ➔ `.customer` ➔ `.address` ➔ `.city`
- If any intermediate key does not exist, n8n provides visual error feedback, helping developers catch missing keys early.

3. Zero-Based Array Indexing

- Arrays in JavaScript and n8n are 0-indexed:
  - `$json.orders[0]` targets the **first** element (`ORD-1001`, total: `149.99`).
  - `$json.orders[1]` targets the **second** element (`ORD-1002`, total: `50.00`).
- Accessing an order total combines bracket indexing with dot notation: `{{ $json.orders[0].total }}`.

---

7. Practical Assessment Checklist

- [x] Workflow created and named **`Topic4_JSON_Data_Assessment`**.
- [x] Manual Trigger node added as initial starting node.

- [x] Code node added and named `Mock Nested JSON Data`.
- [x] Valid nested JSON generated containing customer object, address object, and orders array.
- [x] Orders array contains multiple order objects.
- [x] Output inspected and verified in JSON, Table, and Schema views.
- [x] `customerName` extracted using dot notation (`{{ $json.customer.name }}`).
- [x] `cityOfCustomer` extracted using nested dot notation (`{{ $json.customer.address.city }}`).
- [x] `firstOrderTotal` extracted using array index 0 (`{{ $json.orders[0].total }}`).
- [x] Expressions used throughout without hardcoded values.
- [x] Native String, String, and Number types strictly preserved.
- [x] Final output contains exactly the 3 required fields.
- [x] Workflow executed with zero errors and all green status indicators.
- [x] Exported workflow JSON source file (`Topic4_JSON_Data_Assessment.json`).
- [x] Generated print-ready PDF documentation (`README.pdf`).
- [x] Pushed all files to public GitHub repository.

---

8. LMS Submission Description & Assessor Notes

> **Copy and paste the text below directly into the LMS Description field when submitting:**

```
================================================================================
TAYANA ACADEMY LMS SUBMISSION | TOPIC 4: WORKING WITH JSON OBJECTS, ARRAYS & EXPRESSIONS
================================================================================
Student Name: Abhishek Nikam
Student Email: abhishek.n@tayanasolutions.com
Workflow Name: Topic4_JSON_Data_Assessment
GitHub Repository: https://github.com/3OOM/n8n-assessments/tree/main/topic-04-json-objects-arrays-ex
Demo Video Link: [INSERT YOUR LOOM OR YOUTUBE LINK HERE]

1. WORKFLOW IMPLEMENTATION SUMMARY:
- Constructed a 3-node sequential pipeline: Manual Trigger -> Code (Mock Nested JSON Data) -> Edit F
- "Mock Nested JSON Data" generates a comprehensive nested customer profile:
  * Nested customer object with nested address sub-object (city: "San Francisco")
  * Orders array with multiple items (index 0 total: 149.99, index 1 total: 50.00)
  * Membership status: "Gold"
- "Extract Values" applies targeted n8n expressions to flatten and project data into a simplified ou
  * customerName (String): {{ $json.customer.name }} -> "Jane Doe"
  * cityOfCustomer (String): {{ $json.customer.address.city }} -> "San Francisco"
  * firstOrderTotal (Number): {{ $json.orders[0].total }} -> 149.99
- Final payload contains strictly these 3 properties with clean native types.

2. KEY TECHNICAL HIGHLIGHTS & EXPRESSION SYNTAX:
- Dot Notation Navigation: Accessed customer name and deeply nested address city via $json.customer
- Zero-Based Array Indexing: Extracted the first order total using bracket indexing combined with pr
- Schema Hygiene & Type Preservation: Configured Edit Fields with "Include Other Input Fields: false

3. VERIFICATION ACROSS MULTI-VIEW MODES:
- JSON View: Confirmed valid object syntax with exact required keys.
- Table View: Verified 3 clean tabular columns (customerName, cityOfCustomer, firstOrderTotal).
- Schema View: Confirmed string, string, and number primitive types.

4. ASSUMPTIONS & KNOWN LIMITATIONS:
- Assumes the orders array always contains at least one order object at index 0.
- Production enhancement would utilize optional chaining (e.g. $json.orders?.[0]?.total ?? 0) to gua
================================================================================
```

9. Demo Video Recording Guide (60–90 Seconds)

Timestamp	Screen Action	Voiceover Script
0:00 – 0:15	Show canvas with title `Topic4_JSON_Data_Assessment` and the 3 connected nodes: Manual Trigger ➔ Mock Nested JSON Data ➔ Extract Values.	*"Hello evaluator! In Topic 4, we are exploring JSON objects, arrays, and expression mapping in n8n. Here is our 3-node assessment workflow on the canvas."*
0:15 – 0:35	Open `Mock Nested JSON Data` Code node. Show JavaScript code returning nested customer, address, and orders array. Run node.	*"In the Code node, we generate a nested customer profile containing a nested address object, an orders array with two items, and a membership field. Executing this node, we see our structured JSON."*
0:35 – 0:55	Open `Extract Values` Edit Fields node. Show expressions: `{{ $json.customer.name }}`, `{{ $json.customer.address.city }}`, and `{{ $json.orders[0].total }}`.	*"In 'Extract Values', we extract customerName using dot notation, cityOfCustomer using nested dot notation, and firstOrderTotal using zero-based array indexing orders[0].total."*
0:55 – 1:15	Click **Test workflow**. Open final node output. Toggle Table, JSON, and Schema views. Point out `firstOrderTotal: 149.99`.	*"Running Test workflow, all nodes execute with green checkmarks. In JSON view, we have our clean 3-field output: Jane Doe, San Francisco, and 149.99 as a native Number."*
1:15 – 1:30	Show GitHub repository and PDF documentation. Wrap up.	*"All files including the JSON workflow export and formatted PDF documentation are committed to our public GitHub repo. Thank you!"*